Compiler Construction

Assignment 2: LL(1) Parser Report

Team Members

23i-0832 Humayun Bilal

23i-0728 Muhammad Faheem

1. Introduction

LL(1) parsing is a top-down, predictive parsing technique used to check whether an input string can be generated by a context-free grammar. The parser reads input from left to right, constructs a leftmost derivation, and makes each decision using one lookahead token. Because decisions are table-driven, LL(1) parsing avoids backtracking and is efficient for deterministic grammars.

In this assignment, we built a complete LL(1) parser workflow. The system reads grammar rules from a file, transforms them into LL(1)-compatible form, computes FIRST and FOLLOW sets, builds a predictive parsing table, and parses tokenized input strings using a stack. It also reports syntax errors with recovery and generates parse trees for accepted strings. The objective was to produce a modular and reusable implementation that is correct, readable, and easy to test across different grammars.

2. Approach

2.1 Data Structures Used

- **Grammar representation:** Non-terminal to list of productions (each production as symbol sequence).
- **Sets:** Separate collections for terminals, non-terminals, FIRST, and FOLLOW.
- **LL(1) parsing table:** Non-terminal/lookahead token to selected production mapping.
- **Parsing stack:** Standard stack for predictive parsing operations.
- **Token stream:** List of input tokens for each line (ending with dollar sign $).
- **Parse tree:** Tree nodes with symbol identifier and children list.
- **Error records:** Structure containing line number, position, expected symbol, found symbol, and recovery action.

2.2 Implementation Summary

1. **Grammar loading and normalization:** Read grammar from file and normalize production format.
2. **Grammar transformation for LL(1):** Apply left factoring, remove indirect and direct left recursion.
3. **Set computation:** Compute FIRST sets using iterative fixed-point updates; compute FOLLOW sets with epsilon handling.
4. **Parsing table construction:** Build LL(1) table from FIRST/FOLLOW sets and detect conflicts.
5. **Parsing execution:** Parse input strings using stack and table-driven expansion.
6. **Trace logging:** Log parsing trace at each step for verification and debugging.
7. **Parse tree generation:** Build and output abstract syntax tree for accepted strings.

8. **Error reporting:** Report syntax errors with context and continue parsing for recovery.

2.3 Design Decisions

- **Modularity:** Separate components for grammar transformation, set computation, parsing, tree construction, and error handling.
- **String-based symbols:** Used string representation to support multi-character tokens and epsilon productions.
- **Deterministic parsing:** Table-driven approach with no backtracking for efficiency.
- **Detailed execution trace:** Comprehensive step-by-step logs for verification and debugging.
- **Graceful error recovery:** Panic-mode recovery continues parsing after errors to report multiple issues in a single pass.

2.4 Indirect Left Recursion Handling

We implemented ordered substitution to handle indirect left recursion:

1. Process non-terminals in a fixed order.
2. For each non-terminal A, replace productions A → B α where B is an earlier non-terminal with the productions of B.
3. After substitution completes, remove direct left recursion from the resulting productions using helper non-terminals.

This approach correctly handles recursive dependency chains such as A → B → C → A.

2.5 Error Recovery Strategy

Panic-mode recovery was implemented with the following steps:

1. On mismatch between expected and lookahead symbol, or when parsing table entry is empty, report error with expected symbol, found symbol, line, and token position.
2. **Synchronization:** Pop non-terminals from stack until reaching a symbol whose FOLLOW set contains the lookahead; skip input tokens if necessary.
3. Resume parsing with updated stack and input to detect additional errors.
4. Guarantee progress by ensuring either input consumption or stack reduction on each recovery.

3. Challenges: Difficulties Faced and Solutions

Challenge	Issue	Solution
Epsilon handling in FIRST/FOLLOW	Incorrect epsilon propagation to FOLLOW sets caused wrong table entries and parsing failures.	Implemented fixed-point iteration with explicit epsilon checks in each production scan.
Indirect left recursion	Direct-only left recursion removal failed on mutually recursive rules (e.g., A → B, B → A).	Performed ordered substitution of earlier non-terminals before direct recursion elimination.
Parsing table conflicts	Multiple productions mapped to a single table cell (non-LL(1) grammars).	Added conflict detection and LL(1) validation during table generation with detailed reporting.
Error recovery loops	Parser could repeat the same erroneous state after recovery, causing infinite loops.	Enforced monotonic progress: always consume input or reduce stack on error recovery.
Module integration	Inconsistent conventions across modules caused mismatches in symbol format and epsilon representation.	Standardized symbol format, epsilon representation, and shared interfaces early in development.

4. Test Cases

We tested the parser with **4 different grammars**, each with **5–10 input strings** covering valid, invalid, and edge cases.

Grammar 1: Simple Grammar with Epsilon

File: `input/grammar1.txt`

Grammar:

```
Start -> First Second
First -> a | epsilon
Second -> b
```

Purpose: Test epsilon handling and simple derivations.

Test Inputs (7 strings):

Input	Expected	Purpose
a b	Valid	Both alternatives used
b	Valid	First produces epsilon
a	Invalid	Missing Second (b)
a b b	Invalid	Extra symbol

Input	Expected	Purpose
b a	Invalid	Wrong order
(empty)	Invalid	Missing symbols
a a b	Invalid	Extra symbol in First

Grammar 2: Expression Grammar (Left Recursion)

File: `input/grammar2.txt`

Grammar:

```
Expr   -> Expr + Term | Term
Term   -> Term * Factor | Factor
Factor -> ( Expr ) | id
```

Purpose: Test left recursion removal and operator precedence parsing.

Transformed Grammar (after left recursion removal):

```
Expr      -> Term ExprPrime
ExprPrime -> + Term ExprPrime | epsilon
Term      -> Factor TermPrime
TermPrime -> * Factor TermPrime | epsilon
Factor    -> ( Expr ) | id
```

Test Inputs (10 strings):

Input	Expected	Purpose
id + id * id	Valid	Operator precedence
(id + id) * id	Valid	Parentheses and precedence
id	Valid	Single identifier
id + * id	Invalid	Missing operand before *
(id + id	Invalid	Missing closing paren
id id	Invalid	Missing operator
+ id	Invalid	Missing left operand
id *) id	Invalid	Mismatched paren
id +	Invalid	Missing right operand
( )	Invalid	Empty parentheses

FIRST/FOLLOW Sets:

Non-terminal	FIRST set	FOLLOW set
Expr	{ (, id }	{ $,) }
Term	{ (, id }	{ +, $,) }
Factor	{ (, id }	{ *, +, $,) }
ExprPrime	{ +, ε }	{ $,) }
TermPrime	{ *, ε }	{ +, $,) }

Grammar Status: LL(1) ✓

Grammar 3: Statement Grammar (Left Factoring)

File: `input/grammar3.txt`

Grammar:

```
Stmt -> if Cond then Stmt | if Cond then Stmt else Stmt | a
Cond -> b
```

Purpose: Test left factoring for ambiguous if-then-else constructs.

Transformed Grammar (after left factoring):

```
Stmt  -> if Cond then Stmt Stmt' | a
Stmt' -> else Stmt | epsilon
Cond  -> b
```

Test Inputs (9 strings):

Input	Expected	Purpose
a	Valid	Simple statement
if b then a	Valid	If-then without else
if b then a else a	Valid	If-then-else
if b then if b then a else a	Valid	Nested if-then-else
if b then if b then a	Valid	Nested without else
if then a	Invalid	Missing condition
if b a	Invalid	Missing then

Input	Expected	Purpose
a else a	Invalid	Else without if
if b then	Invalid	Missing statement

Grammar 4: Indirect Left Recursion

File: `input/grammar4.txt`

Grammar:

```
Start -> Alpha a | b
Alpha -> Alpha c | Start d | epsilon
```

Purpose: Test ordered substitution and elimination of indirect left recursion.

Transformed Grammar:

```
Start -> Alpha a | b
Alpha -> (Alpha c | (Alpha a | b) d)* (epsilon | a | epsilon d)
```

Test Inputs (10 strings):

Input	Expected	Purpose
b	Valid	Direct derivation of Start
a	Valid	Start -> Alpha a -> epsilon a
c a	Valid	Alpha c derivation
b d a	Valid	Indirect recursion: Start d then a
a d a	Invalid	Multiple alphas incorrectly
c c a	Valid	Multiple c's
d a	Invalid	Missing main derivation
b a	Valid	b d -> a alternative
a d	Invalid	Missing final a
c	Invalid	Incomplete production

5. Verification

We verified correctness through multiple approaches:

1. Grammar Transformation Validation

- Confirmed left factoring eliminates common prefixes.
- Verified left recursion removal produces equivalent grammar without recursion.
- Checked that epsilon productions are correctly placed.

2. FIRST/FOLLOW Set Verification

- Manually computed sets for small grammars and compared with implementation output.
- Verified epsilon is correctly handled in FIRST propagation.
- Confirmed FOLLOW sets include all tokens that can follow a non-terminal.

3. Parsing Table Consistency

- Ensured each (non-terminal, lookahead) pair maps to exactly one production or is empty.
- Verified LL(1) property: no conflicts detected for transformed grammars.
- Checked that all productions appear in the table where applicable.

4. Parsing Correctness

- Traced parse execution step-by-step for valid inputs and verified tree structure.
- Confirmed invalid inputs are rejected with appropriate error messages.
- Verified error recovery continues parsing and reports multiple errors.

5. Parse Tree Validation

- Checked that generated trees match grammar structure.
- Verified tree leaves correspond to input tokens.
- Confirmed parse tree respects left-to-right input order.

6. Sample Outputs

Example 1: Successful Parse

Grammar 2 — Input: `id + id * id`

Parsing Trace:

```
Step   | Stack                           | Input                   | Action
------|--------------------------------|------------------------|------------------
---------
1      | $ Expr                          | id + id * id $          | Expr -> Term
ExprPrime
2      | $ ExprPrime Term                | id + id * id $          | Term -> Factor
TermPrime
3      | $ ExprPrime TermPrime Factor    | id + id * id $          | Factor -> id
4      | $ ExprPrime TermPrime id        | id + id * id $          | Match id
5      | $ ExprPrime TermPrime           | + id * id $             | TermPrime ->
epsilon
6      | $ ExprPrime                     | + id * id $             | ExprPrime -> +
Term ExprPrime
7      | $ ExprPrime Term +              | + id * id $             | Match +
8      | $ ExprPrime Term                | id * id $               | Term -> Factor
TermPrime
```

```
9     | $ ExprPrime TermPrime Factor   | id * id $              | Factor -> id
10    | $ ExprPrime TermPrime id       | id * id $              | Match id
11    | $ ExprPrime TermPrime          | * id $                 | TermPrime -> *
Factor TermPrime
12    | $ ExprPrime TermPrime Factor * | * id $                 | Match *
13    | $ ExprPrime TermPrime Factor   | id $                   | Factor -> id
14    | $ ExprPrime TermPrime id       | id $                   | Match id
15    | $ ExprPrime TermPrime          | $                      | TermPrime ->
epsilon
16    | $ ExprPrime                    | $                      | ExprPrime ->
epsilon
17    | $                              | $                      | Accept

Result: String accepted successfully!
```

Parse Tree (ASCII):

```
`-- Expr
    |-- Term
    |   |-- Factor
    |   |   `-- id
    |   `-- TermPrime
    |       `-- epsilon
    `-- ExprPrime
        |-- +
        |-- Term
        |   |-- Factor
        |   |   `-- id
        |   `-- TermPrime
        |       |-- *
        |       |-- Factor
        |       |   `-- id
        |       `-- TermPrime
        |           `-- epsilon
        `-- ExprPrime
            `-- epsilon
```

Example 2: Error Detection and Recovery

Grammar 2 — Input: `id + * id`

Parsing Trace with Error:

```
Step  | Stack                          | Input        | Action
------|--------------------------------|--------------|----------------------------
---------
1     | $ Expr                         | id + * id $  | Expr -> Term ExprPrime
2     | $ ExprPrime Term               | id + * id $  | Term -> Factor TermPrime
3     | $ ExprPrime TermPrime Factor   | id + * id $  | Factor -> id
4     | $ ExprPrime TermPrime id       | id + * id $  | Match id
5     | $ ExprPrime TermPrime          | + * id $     | TermPrime -> epsilon
6     | $ ExprPrime                    | + * id $     | ExprPrime -> + Term
ExprPrime
7     | $ ExprPrime Term +             | + * id $     | Match +
8     | $ ExprPrime Term               | * id $       | Term -> Factor TermPrime
9     | $ ExprPrime TermPrime Factor   | * id $       | Factor -> ?

ERROR: Expected 'id' or '(', found '*' at position 4
Recovery: Skipping '*', attempting to continue...
```

```
10    | $ ExprPrime TermPrime Factor  | id $          | Factor -> id
11    | $ ExprPrime TermPrime id     | id $          | Match id
12    | $ ExprPrime TermPrime        | $             | TermPrime -> epsilon
13    | $ ExprPrime                  | $             | ExprPrime -> epsilon
14    | $                            | $             | End of input

Result: String rejected with 1 error recovered.
```

Example 3: FIRST/FOLLOW Sets (Grammar 2)

Output from `first_follow_sets.txt`**:**

```
FIRST Sets
FIRST(Expr)         = { (, id }
FIRST(Term)         = { (, id }
FIRST(Factor)       = { (, id }
FIRST(ExprPrime)    = { +, epsilon }
FIRST(TermPrime)    = { *, epsilon }

FOLLOW Sets
FOLLOW(Expr)        = { $, ) }
FOLLOW(Term)        = { +, $, ) }
FOLLOW(Factor)      = { *, +, $, ) }
FOLLOW(ExprPrime)   = { $, ) }
FOLLOW(TermPrime)   = { +, $, ) }
```

Example 4: Grammar Transformation (Left Recursion Removal)

Original Grammar (Grammar 2):

```
Expr   -> Expr + Term | Term
Term   -> Term * Factor | Factor
Factor -> ( Expr ) | id
```

Transformed Grammar (`output/grammar_transformed.txt`**):**

```
Expr      -> Term ExprPrime
ExprPrime -> + Term ExprPrime | epsilon
Term      -> Factor TermPrime
TermPrime -> * Factor TermPrime | epsilon
Factor    -> ( Expr ) | id
```

The transformation removes direct left recursion by introducing helper non-terminals (ExprPrime, TermPrime) that handle the optional repetition tail.

7. Conclusion

This assignment demonstrated the key steps of LL(1) parser construction: grammar normalization, FIRST/FOLLOW computation, LL(1) table generation, and deterministic parsing with error recovery. The implementation correctly handles:

- **Left recursion elimination:** Both direct and indirect cases using ordered substitution.

- **Left factoring:** Removal of common prefixes to resolve ambiguities.
- **Epsilon productions:** Proper handling in FIRST and FOLLOW set computation.
- **Parsing and tree generation:** Deterministic table-driven parsing with complete parse trees.
- **Error recovery:** Panic-mode recovery continues parsing to detect multiple errors.

Key lessons learned:

1. **Transformation correctness is critical:** Incorrect grammar transformation cascades into wrong FIRST/FOLLOW sets and parsing failures.
2. **Epsilon handling requires care:** Fixed-point iteration and explicit checks are necessary to avoid propagation errors.
3. **Modular design aids debugging:** Separating concerns allows testing each component independently.
4. **Error recovery improves usability:** Users benefit from seeing all syntax errors in one pass rather than stopping at the first error.
5. **Comprehensive tracing is invaluable:** Detailed step-by-step logs facilitate verification and debugging of complex parsing scenarios.

The resulting parser is robust, maintainable, and provides informative feedback for both successful parses and error cases. It serves as a foundation for more advanced parsing techniques and compiler construction tasks.